

7.2 The Resource Manager

Every game is constructed from a wide variety of *resources* (sometimes called *assets* or *media*). Examples include meshes, materials, textures, shader programs, animations, audio clips, level layouts, collision primitives, physics parameters, and the list goes on. A game's resources must be managed, both in terms of the offline tools used to create them, and in terms of loading, unloading and manipulating them at runtime. Therefore, every game engine has a *resource manager* of some kind.

Every resource manager is comprised of two distinct but integrated components. One component manages the chain of offline tools used to create the assets and transform them into their engine-ready form. The other component manages the resources at runtime, ensuring that they are loaded into memory in advance of being needed by the game and making sure they are unloaded from memory when no longer needed.

In some engines, the resource manager is a cleanly designed, unified, centralized subsystem that manages all types of resources used by the game. In other engines, the resource manager doesn't exist as a single subsystem per se, but rather is spread across a disparate collection of subsystems, perhaps written by different individuals at various times over the engine's long and sometimes colorful history. But no matter how it is implemented, a resource manager invariably takes on certain responsibilities and solves a well-understood set of problems. In this section, we'll explore the functionality and some of the implementation details of a typical game engine resource manager.

7.2.1 Offline Resource Management and the Tool Chain

7.2.1.1 Revision Control for Assets

On a small game project, the game's assets can be managed by keeping loose files sitting around on a shared network drive with an ad hoc directory structure. This approach is not feasible for a modern commercial 3D game, comprised of a massive number and variety of assets. For such a project, the team requires a more formalized way to track and manage its assets.

Some game teams use a source code revision control system to manage their resources. Art source files (Maya scenes, Photoshop PSD files, Illustrator files, etc.) are checked in to Perforce or a similar package by the artists. This approach works reasonably well, although some game teams build custom asset management tools to help flatten the learning curve for their artists. Such tools may be simple wrappers around a commercial revision control system, or they might be entirely custom.

Dealing with Data Size

One of the biggest problems in the revision control of art assets is the sheer amount of data. Whereas C++ and script source code files are small, relative to their impact on the project, art files tend to be much, much larger. Because many source control systems work by copying files from the central repository down to the user's local machine, the sheer size of the asset files can render these packages almost entirely useless.

I've seen a number of different solutions to this problem employed at various studios. Some studios turn to commercial revision control systems like Alienbrain that have been specifically designed to handle very large data sizes. Some teams simply "take their lumps" and allow their revision control tool to copy assets locally. This can work, as long as your disks are big enough and your network bandwidth sufficient, but it can also be inefficient and slow the team down. Some teams build elaborate systems on top of their revision control tool to ensure that a particular end user only gets local copies of the files he or she actually needs. In this model, the user either has no access to the rest of the repository or can access it on a shared network drive when needed.

At Naughty Dog we use a proprietary tool that makes use of UNIX symbolic links to virtually eliminate data copying, while permitting each user to have a complete local view of the asset repository. As long as a file is not checked out for editing, it is a symlink to a master file on a shared network drive. A symbolic link occupies very little space on the local disk, because it is nothing more than a directory entry. When the user checks out a file for editing, the symlink is removed, and a local copy of the file replaces it. When the user is done editing and checks the file in, the local copy becomes the new master copy, its revision history is updated in a master database, and the local file turns back into a symlink. This system works very well, but it requires the team to build their own revision control system from scratch; I am unaware of any commercial tool that works like this. Also, symbolic links are a UNIX feature—such a tool could probably be built with Windows junctions (the Windows equivalent of a symbolic link), but I haven't seen anyone try it as yet.

7.2.1.2 The Resource Database

As we'll explore in depth in the next section, most assets are not used in their original format by the game engine. They need to pass through some kind of asset conditioning pipeline, whose job it is to convert the assets into the binary format needed by the engine. For every resource that passes through

the asset conditioning pipeline, there is some amount of *metadata* that describes *how* that resource should be processed. When compressing a texture bitmap, we need to know what *type* of compression best suits that particular image. When exporting an animation, we need to know what range of frames in Maya should be exported. When exporting character meshes out of a Maya scene containing multiple characters, we need to know which mesh corresponds to which character in the game.

To manage all of this metadata, we need some kind of database. If we are making a very small game, this database might be housed in the brains of the developers themselves. I can hear them now: “Remember: the player’s animations need to have the ‘flip X’ flag set, but the other characters must *not* have it set...or...rats...is it the other way around?”

Clearly for any game of respectable size, we simply cannot rely on the memories of our developers in this manner. For one thing, the sheer volume of assets becomes overwhelming quite quickly. Processing individual resource files by hand is also far too time-consuming to be practical on a full-fledged commercial game production. Therefore, every professional game team has some kind of semiautomated resource pipeline, and the data that drive the pipeline is stored in some kind of *resource database*.

The resource database takes on vastly different forms in different game engines. In one engine, the metadata describing how a resource should be built might be embedded into the source assets themselves (e.g., it might be stored as so-called blind data within a Maya file). In another engine, each source resource file might be accompanied by a small text file that describes how it should be processed. Still other engines encode their resource building metadata in a set of XML files, perhaps wrapped in some kind of custom graphical user interface. Some engines employ a true relational database, such as Microsoft Access, MySQL or conceivably even a heavyweight database like Oracle.

Whatever its form, a resource database must provide the following basic functionality:

- The ability to deal with multiple *types* of resources, ideally (but certainly not necessarily) in a somewhat consistent manner.
- The ability to create new resources.
- The ability to delete resources.
- The ability to inspect and modify existing resources.
- The ability to move a resource’s source file(s) from one location to another on-disk. (This is very helpful because artists and game designers

often need to rearrange assets to reflect changing project goals, rethinking of game designs, feature additions and cuts, etc.)

- The ability of a resource to cross-reference other resources (e.g., the material used by a mesh, or the collection of animations needed by level 17). These cross-references typically drive both the resource building process and the loading process at runtime.
- The ability to maintain *referential integrity* of all cross-references within the database and to do so in the face of all common operations such as deleting or moving resources around.
- The ability to maintain a revision history, complete with a log of who made each change and why.
- It is also very helpful if the resource database supports searching or querying in various ways. For example, a developer might want to know in which levels a particular animation is used or which textures are referenced by a set of materials. Or they might simply be trying to find a resource whose name momentarily escapes them.

It should be pretty obvious from looking at the above list that creating a reliable and robust resource database is no small task. When designed well and implemented properly, the resource database can quite literally make the difference between a team that ships a hit game and a team that spins its wheels for 18 months before being forced by management to abandon the project (or worse). I know this to be true, because I've personally experienced both.

7.2.1.3 Some Successful Resource Database Designs

Every game team will have different requirements and make different decisions when designing their resource database. However, for what it's worth, here are some designs that have worked well in my own experience.

Unreal Engine 4

Unreal's resource database is managed by their über-tool, UnrealEd. UnrealEd is responsible for literally everything, from resource metadata management to asset creation to level layout and more. UnrealEd has its drawbacks, but its single biggest benefit is that UnrealEd is a part of the game engine itself. This permits assets to be created and then immediately viewed in their full glory, exactly as they will appear in-game. The game can even be run from within UnrealEd in order to visualize the assets in their natural surroundings and see if and how they work in-game.

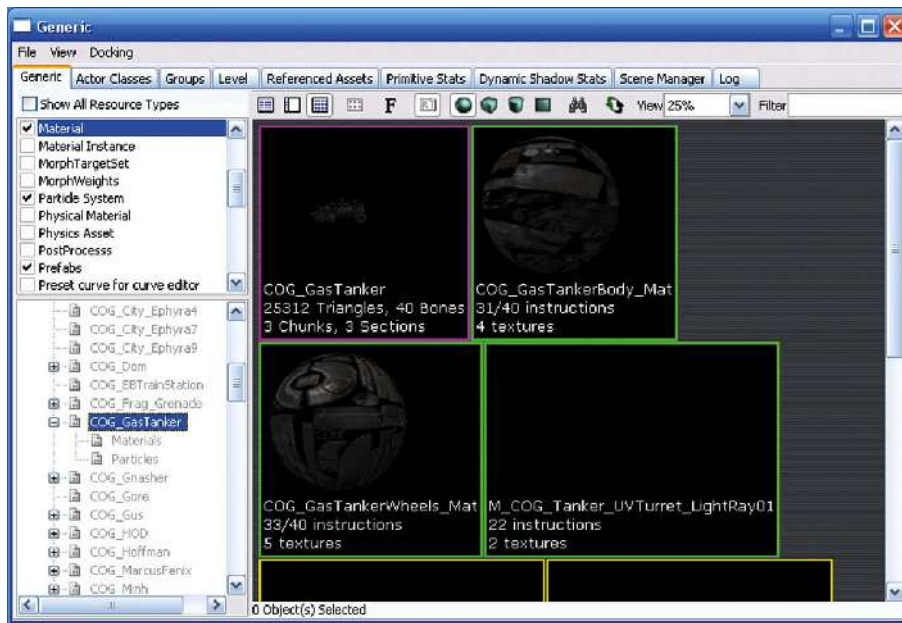


Figure 7.1. UnrealEd's Generic Browser.

Another big benefit of UnrealEd is what I would call *one-stop shopping*. UnrealEd's Generic Browser (depicted in Figure 7.1) allows a developer to access literally every resource that is consumed by the engine. Having a single, unified and reasonably consistent interface for creating and managing all types of resources is a big win. This is especially true considering that the resource data in most other game engines is fragmented across countless inconsistent and often cryptic tools. Just being able to *find* any resource easily in UnrealEd is a big plus.

Unreal can be less error-prone than many other engines, because assets must be explicitly imported into Unreal's resource database. This allows resources to be checked for validity very early in the production process. In most game engines, any old data can be thrown into the resource database, and you only know whether or not that data is valid when it is eventually built—or sometimes not until it is actually loaded into the game at runtime. But with Unreal, assets can be validated as soon as they are imported into UnrealEd. This means that the person who created the asset gets immediate feedback as to whether his or her asset is configured properly.

Of course, Unreal's approach has some serious drawbacks. For one thing,

all resource data is stored in a small number of large package files. These files are binary, so they are not easily merged by a revision control package like CVS, Subversion or Perforce. This presents some major problems when more than one user wants to modify resources that reside in a single package. Even if the users are trying to modify *different* resources, only one user can lock the package at a time, so the other has to wait. The severity of this problem can be reduced by dividing resources into relatively small, granular packages, but it cannot practically be eliminated.

Referential integrity is quite good in UnrealEd, but there are still some problems. When a resource is renamed or moved around, all references to it are maintained automatically using a dummy object that remaps the old resource to its new name/location. The problem with these dummy remapping objects is that they hang around and accumulate and sometimes cause problems, especially if a resource is deleted. Overall, Unreal's referential integrity is quite good, but it is not perfect.

Despite its problems, UnrealEd is by far the most user-friendly, well-integrated and streamlined asset creation toolkit, resource database and asset conditioning pipeline that I have ever worked with.

Naughty Dog's Engine

For *Uncharted: Drake's Fortune*, Naughty Dog stored its resource metadata in a MySQL database. A custom graphical user interface was written to manage the contents of the database. This tool allowed artists, game designers and programmers alike to create new resources, delete existing resources and inspect and modify resources as well. This GUI was a crucial component of the system, because it allowed users to avoid having to learn the intricacies of interacting with a relational database via SQL.

The original MySQL database used on *Uncharted* did not provide a useful history of the changes made to the database, nor did it provide a good way to roll back "bad" changes. It also did not support multiple users editing the same resource, and it was difficult to administer. Naughty Dog has since moved away from MySQL in favor of an XML file-based asset database, managed under Perforce.

Builder, Naughty Dog's resource database GUI, is depicted in Figure 7.2. The window is broken into two main sections: a tree view showing all resources in the game on the left and a properties window on the right, allowing the resource(s) that are selected in the tree view to be viewed and edited. The resource tree contains folders for organizational purposes, so that the artists and game designers can organize their resources in any way they see fit. Various types of resources can be created and managed within any folder, includ-

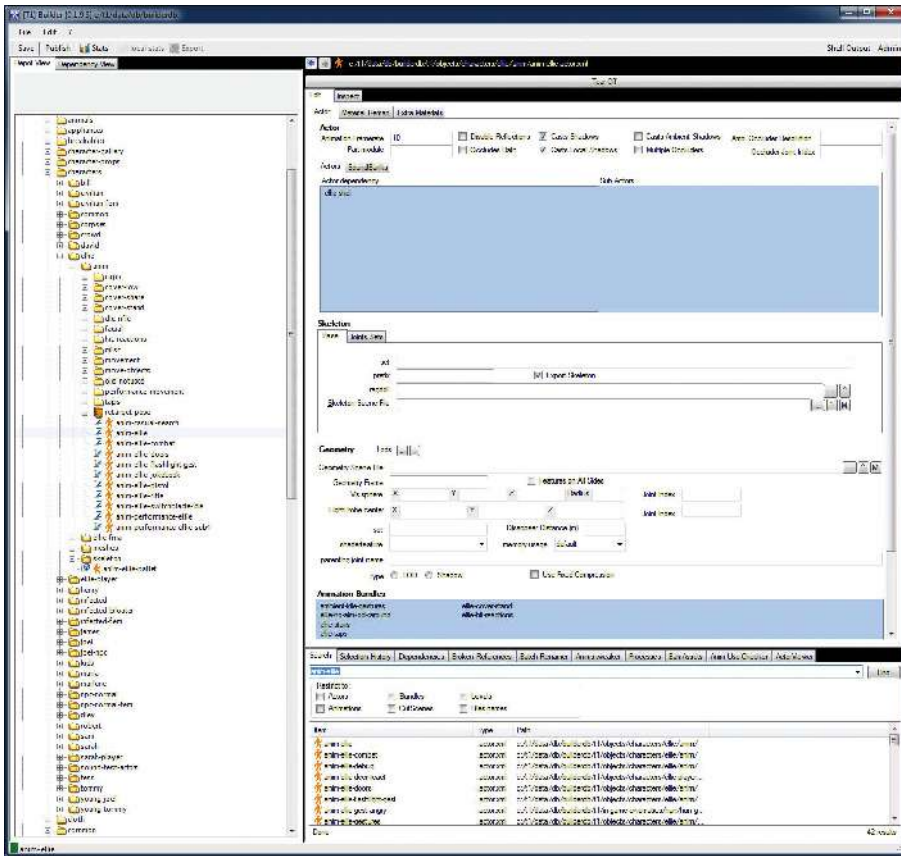


Figure 7.2. The front-end GUI for Naughty Dog's offline resource database, Builder.

ing actors and levels, and the various subresources that comprise them (primarily meshes, skeletons and animations). Animations can also be grouped into pseudo-folders known as bundles. This allows large groups of animations to be created and then managed as a unit, and prevents a lot of wasted time dragging individual animations around in the tree view.

The asset conditioning pipeline employed on the *Uncharted* and *The Last of Us* series consists of a set of resource exporters, compilers and linkers that are run from the command line. The engine is capable of dealing with a wide variety of different kinds of data objects, but these are packaged into one of two types of resource files: actors and levels. An actor can contain skeletons, meshes, materials, textures and/or animations. A level contains static background meshes, materials and textures, and also level-layout information. To

build an actor, one simply types `ba name-of-actor` on the command line; to build a level, one types `b1 name-of-level`. These command-line tools query the database to determine exactly *how* to build the actor or level in question. This includes information on how to export the assets from DCC tools like Maya and Photoshop, how to process the data, and how to package it into binary `.pak` files that can be loaded by the game engine. This is much simpler than in many engines, where resources have to be *exported manually* by the artists—a time-consuming, tedious and error-prone task.

The benefits of the resource pipeline design used by Naughty Dog include:

- *Granular resources.* Resources can be manipulated in terms of logical entities in the game—meshes, materials, skeletons and animations. These resource types are granular enough that the team almost never has conflicts in which two users want to edit the same resource simultaneously.
- *The necessary features (and no more).* The Builder tool provides a powerful set of features that meet the needs of the team, but Naughty Dog didn't waste any resources creating features they didn't need.
- *Obvious mapping to source files.* A user can very quickly determine which source assets (native DCC files, like Maya `.ma` files or photoshop `.psd` files) make up a particular resource.
- *Easy to change how DCC data is exported and processed.* Just click on the resource in question and twiddle its processing properties within the resource database GUI.
- *Easy to build assets.* Just type `ba` or `b1` followed by the resource name on the command line. The dependency system takes care of the rest.

Of course, Naughty Dog's tool chain does have some drawbacks as well, including:

- *Lack of visualization tools.* The only way to preview an asset is to load it into the game or the model/animation viewer (which is really just a special mode of the game itself).
- *The tools aren't fully integrated.* Naughty Dog uses one tool to lay out levels, another to manage the majority of resources in the resource database, and a third to set up materials and shaders (this is not part of the resource database front end). Building the assets is done on the command line. It might be a bit more convenient if all of these functions were to be integrated into a single tool. However, Naughty Dog has no plans to do this, because the benefit would probably not outweigh the costs involved.

OGRE's Resource Manager System

OGRE is a rendering engine, not a full-fledged game engine. That said, OGRE does boast a reasonably complete and very well-designed runtime resource manager. A simple, consistent interface is used to load virtually any kind of resource. And the system has been designed with extensibility in mind. Any programmer can quite easily implement a resource manager for a brand new kind of asset and integrate it easily into OGRE's resource framework.

One of the drawbacks of OGRE's resource manager is that it is a runtime-only solution. OGRE lacks any kind of offline resource database. OGRE does provide some exporters that are capable of converting a Maya file into a mesh that can be used by OGRE (complete with materials, shaders, a skeleton and optional animations). However, the exporter must be run manually from within Maya itself. Worse, all of the metadata describing how a particular Maya file should be exported and processed must be entered by the user doing the export.

In summary, OGRE's runtime resource manager is powerful and well-designed. But, OGRE would benefit a great deal from an equally powerful and modern resource database and asset conditioning pipeline on the tools side.

Microsoft's XNA

XNA is a game development toolkit by Microsoft, targeted at the PC and Xbox 360 platforms. Although it was retired by Microsoft in 2014, it's still a good resource for learning about game engines. XNA's resource management system is unique, in that it leverages the project management and build systems of the Visual Studio IDE to manage and build the assets in the game as well. XNA's game development tool, Game Studio Express, is just a plug-in to Visual Studio Express.

7.2.1.4 The Asset Conditioning Pipeline

In Section 1.7, we learned that resource data is typically created using advanced digital content creation (DCC) tools like Maya, ZBrush, Photoshop or Houdini. However, the data formats used by these tools are usually not suitable for direct consumption by a game engine. So the majority of resource data is passed through an *asset conditioning pipeline* (ACP) on its way to the game engine. The ACP is sometimes referred to as the *resource conditioning pipeline* (RCP), or simply the *tool chain*.

Every resource pipeline starts with a collection of *source assets* in native DCC formats (e.g., Maya .ma or .mb files, Photoshop .psd files, etc.). These

assets are typically passed through three processing stages on their way to the game engine:

1. *Exporters.* We need some way of getting the data out of the DCC's native format and into a format that we can manipulate. This is usually accomplished by writing a custom plug-in for the DCC in question. It is the plug-in's job to export the data into some kind of intermediate file format that can be passed to later stages in the pipeline. Most DCC applications provide a reasonably convenient mechanism for doing this. Maya actually provides three: a C++ SDK, a scripting language called MEL and most recently a Python interface as well.

In cases where a DCC application provides no customization hooks, we can always save the data in one of the DCC tool's native formats. With any luck, one of these will be an open format, a reasonably intuitive text format, or some other format that we can reverse engineer. Presuming this is the case, we can pass the file directly to the next stage of the pipeline.

2. *Resource compilers.* We often have to "massage" the raw data exported from a DCC application in various ways in order to make them game-ready. For example, we might need to rearrange a mesh's triangles into strips, or compress a texture bitmap, or calculate the arc lengths of the segments of a Catmull-Rom spline. Not all types of resources need to be compiled—some might be game-ready immediately upon being exported.
3. *Resource linkers.* Multiple resource files sometimes need to be combined into a single useful package prior to being loaded by the game engine. This mimics the process of linking together the object files of a compiled C++ program into an executable file, and so this process is sometimes called *resource linking*. For example, when building a complex composite resource like a 3D model, we might need to combine the data from multiple exported mesh files, multiple material files, a skeleton file and multiple animation files into a single resource. Not all types of resources need to be linked—some assets are game-ready after the export or compile steps.

Resource Dependencies and Build Rules

Much like compiling the source files in a C or C++ project and then linking them into an executable, the asset conditioning pipeline processes source assets (in the form of Maya geometry and animation files, Photoshop PSD files,

raw audio clips, text files, etc.), converts them into game-ready form and then links them together into a cohesive whole for use by the engine. And just like the source files in a computer program, game assets often have interdependencies. (For example, a mesh refers to one or more materials, which in turn refer to various textures.) These interdependencies typically have an impact on the order in which assets must be processed by the pipeline. (For example, we might need to build a character's skeleton before we can process any of that character's animations.) In addition, the dependencies between assets tell us which assets need to be rebuilt when a particular source asset changes.

Build dependencies revolve not only around changes to the assets themselves, but also around changes to data formats. If the format of the files used to store triangle meshes changes, for instance, all meshes in the entire game may need to be reexported and/or rebuilt. Some game engines employ data formats that are robust to version changes. For example, an asset may contain a version number, and the game engine may include code that "knows" how to load and make use of legacy assets. The downside of such a policy is that asset files and engine code tend to become bulky. When data format changes are relatively rare, it may be better to just bite the bullet and reprocess all the files when format changes do occur.

Every asset conditioning pipeline requires a set of rules that describe the interdependencies between the assets, and some kind of build tool that can use this information to ensure that the proper assets are built, in the proper order, when a source asset is modified. Some game teams roll their own build system. Others use an established tool, such as `make`. Whatever solution is selected, teams should treat their build dependency system with utmost care. If you don't, changes to sources, assets may not trigger the proper assets to be rebuilt. The result can be inconsistent game assets, which may lead to visual anomalies or even engine crashes. In my personal experience, I've witnessed countless hours wasted in tracking down problems that could have been avoided had the asset interdependencies been properly specified and the build system implemented to use them reliably.

7.2.2 Runtime Resource Management

Let us turn our attention now to how the assets in our resource database are loaded, managed and unloaded within the engine at runtime.

7.2.2.1 Responsibilities of the Runtime Resource Manager

A game engine's runtime resource manager takes on a wide range of responsibilities, all related to its primary mandate of loading resources into memory:

- Ensures that only *one copy* of each unique resource exists in memory at any given time.
- Manages the *lifetime* of each resource.
- *Loads* needed resources and *unloads* resources that are no longer needed.
- Handles loading of *composite resources*. A composite resource is a resource comprised of other resources. For example, a *3D model* is a composite resource that consists of a mesh, one or more materials, one or more textures and optionally a skeleton and multiple skeletal animations.
- Maintains *referential integrity*. This includes *internal* referential integrity (cross-references within a single resource) and *external* referential integrity (cross-references between resources). For example, a model refers to its mesh and skeleton; a mesh refers to its materials, which in turn refer to texture resources; animations refer to a skeleton, which ultimately ties them to one or more models. When loading a composite resource, the resource manager must ensure that all necessary subresources are loaded, and it must patch in all of the cross-references properly.
- Manages the *memory usage* of loaded resources and ensures that resources are stored in the appropriate place(s) in memory.
- Permits *custom processing* to be performed on a resource after it has been loaded, on a per-resource-type basis. This process is sometimes known as *logging in* or *load-initializing* the resource.
- Usually (but not always) provides a single *unified interface* through which a wide variety of resource types can be managed. Ideally a resource manager is also easily extensible, so that it can handle new types of resources as they are needed by the game development team.
- Handles *streaming* (i.e., asynchronous resource loading), if the engine supports this feature.

7.2.2.2 Resource File and Directory Organization

In some game engines (typically PC engines), each individual resource is managed in a separate “loose” file on-disk. These files are typically contained within a tree of directories whose internal organization is designed primarily for the convenience of the people creating the assets; the engine typically doesn’t care where resource files are located within the resource tree. Here’s a typical resource directory tree for a hypothetical game called *Space Evaders*:

SpaceEvaders	Root directory for entire game.
Resources	Root of all resources.
NPC	Non-player character models and animations.
Pirate	Models and animations for pirates.
Marine	Models and animations for marines.
...	
Player	Player character models and animations.
Weapons	Models and animations for weapons.
Pistol	Models and animations for the pistol.
Rifle	Models and animations for the rifle.
BFG	Models and animations for the big...uh...gun.
...	
Levels	Background geometry and level layouts.
Level1	First level's resources.
Level2	Second level's resources.
...	
Objects	Miscellaneous 3D objects.
Crate	The ubiquitous breakable crate.
Barrel	The ubiquitous exploding barrel.

Other engines package multiple resources together in a single file, such as a ZIP archive, or some other composite file (perhaps of a proprietary format). The primary benefit of this approach is improved load times. When loading data from files, the three biggest costs are *seek times* (i.e., moving the read head to the correct place on the physical media), the time required to open each individual file, and the time to read the data from the file into memory. Of these, the seek times and file-open times can be nontrivial on many operating systems. When a single large file is used, all of these costs are minimized. A single file can be organized sequentially on the disk, reducing seek times to a minimum. And with only one file to open, the cost of opening individual resource files is eliminated.

Solid-state drives (SSD) do not suffer from the seek time problems that plague spinning media like DVDs, Blu-ray discs and hard disc drives (HDD). However, no game console to date includes a solid-state drive as the primary fixed storage device (not even the PS4 and Xbox One). So designing your game's I/O patterns in order to minimize seek times is likely to be a necessity for some time to come.

The OGRE rendering engine's resource manager permits resources to exist

as loose files on disk, or as virtual files within a large ZIP archive. The primary benefits of the ZIP format are the following:

1. *It is an open format.* The `zlib` and `zzip` libraries used to read and write ZIP archives are freely available. The `zlib` SDK is totally free (see <http://www.zlib.net>), while the `zzip` SDK falls under the Lesser Gnu Public License (LGPL) (see <http://zzip.sourceforge.net>).
2. *The virtual files within a ZIP archive “remember” their relative paths.* This means that a ZIP archive “looks like” a raw file system for most intents and purposes. The OGRE resource manager identifies all resources uniquely via strings that appear to be file system paths. However, these paths sometimes identify virtual files within a ZIP archive instead of loose files on disk, and a game programmer needn’t be aware of the difference in most situations.
3. *ZIP archives may be compressed.* This reduces the amount of disk space occupied by resources. But, more importantly, it again speeds up load times, as less data need be loaded into memory from the fixed disk. This is especially helpful when reading data from a DVD-ROM or Blu-ray disk, as the data transfer rates of these devices are much slower than a hard disk drive. Hence the cost of decompressing the data after it has been loaded into memory is often more than offset by the time saved in loading less data from the device.
4. *ZIP archives are modular.* Resources can be grouped together into a ZIP file and managed as a unit. One particularly elegant application of this idea is in product localization. All of the assets that need to be localized (such as audio clips containing dialogue and textures that contain words or region-specific symbols) can be placed in a single ZIP file, and then different versions of this ZIP file can be generated, one for each language or region. To run the game for a particular region, the engine simply loads the corresponding version of the ZIP archive.

The Unreal Engine takes a similar approach, with a few important differences. In Unreal, all resources must be contained within large composite files known as *packages* (a.k.a. “pak files”). No loose disk files are permitted. The format of a package file is proprietary. The Unreal Engine’s game editor, UnrealEd, allows developers to create and manage packages and the resources they contain.

7.2.2.3 Resource File Formats

Each type of resource file potentially has a different format. For example, a mesh file is always stored in a different format than that of a texture bitmap. Some kinds of assets are stored in standardized, open formats. For example, textures are typically stored as Targa files (TGA), Portable Network Graphics files (PNG), Tagged Image File Format files (TIFF), Joint Photographic Experts Group files (JPEG) or Windows Bitmap files (BMP)—or in a standardized compressed format such as DirectX's S3 Texture Compression family of formats (S3TC, also known as DXT n or DXTC). Likewise, 3D mesh data is often exported out of a modeling tool like Maya or Lightwave into a standardized format such as OBJ or COLLADA for consumption by the game engine.

Sometimes a single file format can be used to house many different types of assets. For example, the *Granny* SDK by Rad Game Tools (<http://www.radgametools.com>) implements a flexible open file format that can be used to store 3D mesh data, skeletal hierarchies and skeletal animation data. (In fact the Granny file format can be easily repurposed to store virtually any kind of data imaginable.)

Many game engine programmers roll their own file formats for various reasons. This might be necessary if no standardized format provides all of the information needed by the engine. Also, many game engines endeavor to do as much offline processing as possible in order to minimize the amount of time needed to load and process resource data at runtime. If the data needs to conform to a particular layout in memory, for example, a raw binary format might be chosen so that the data can be laid out by an offline tool (rather than attempting to format it at runtime after the resource has been loaded).

7.2.2.4 Resource GUIDs

Every resource in a game must have some kind of *globally unique identifier* (GUID). The most common choice of GUID is the resource's file system path (stored either as a string or a 32-bit hash). This kind of GUID is intuitive, because it clearly maps each resource to a physical file on-disk. And it's guaranteed to be unique across the entire game, because the operating system already guarantees that no two files will have the same path.

However, a file system path is by no means the only choice for a resource GUID. Some engines use a less-intuitive type of GUID, such as a 128-bit hash code, perhaps assigned by a tool that guarantees uniqueness. In other engines, using a file system path as a resource identifier is infeasible. For example, Unreal Engine stores many resources in a single large file known as a *package*, so the path to the package file does not uniquely identify any one resource. To overcome this problem, an Unreal package file is organized into a folder

hierarchy containing individual resources. Unreal gives each individual resource within a package a unique name, which looks much like a file system path. So in Unreal, a resource GUID is formed by concatenating the (unique) name of the package file with the in-package path of the resource in question. For example, the *Gears of War* resource GUID `Locust_Boomer.PhysicalMaterials.LocustBoomerLeather` identifies a material called `LocustBoomerLeather` within the `PhysicalMaterials` folder of the `Locust_Boomer` package file.

7.2.2.5 The Resource Registry

In order to ensure that only one copy of each unique resource is loaded into memory at any given time, most resource managers maintain some kind of *registry* of loaded resources. The simplest implementation is a *dictionary*—i.e., a collection of *key-value pairs*. The keys contain the unique ids of the resources, while the values are typically pointers to the resources in memory.

Whenever a resource is loaded into memory, an entry for it is added to the resource registry dictionary, using its GUID as the key. Whenever a resource is unloaded, its registry entry is removed. When a resource is requested by the game, the resource manager looks up the resource by its GUID within the resource registry. If the resource can be found, a pointer to it is simply returned. If the resource cannot be found, it can either be loaded automatically or a failure code can be returned.

At first blush, it might seem most intuitive to automatically load a requested resource if it cannot be found in the resource registry. And in fact, some game engines do this. However, there are some serious problems with this approach. Loading a resource is a slow operation, because it involves locating and opening a file on disk, reading a potentially large amount of data into memory (from a potentially slow device like a DVD-ROM drive), and also possibly performing post-load initialization of the resource data once it has been loaded. If the request comes during active gameplay, the time it takes to load the resource might cause a very noticeable hitch in the game's frame rate, or even a multi-second freeze. For this reason, engines tend to take one of two alternative approaches:

1. Resource loading might be disallowed completely during active gameplay. In this model, all of the resources for a game level are loaded *en masse* just prior to gameplay, usually while the player watches a loading screen or progress bar of some kind.
2. Resource loading might be done *asynchronously* (i.e., the data might be *streamed*). In this model, while the player is engaged in level X, the re-

sources for level Y are being loaded in the background. This approach is preferable because it provides the player with a load-screen-free play experience. However, it is considerably more difficult to implement.

7.2.2.6 Resource Lifetime

The *lifetime* of a resource is defined as the time period between when it is first loaded into memory and when its memory is reclaimed for other purposes. One of the resource manager's jobs is to manage resource lifetimes—either automatically or by providing the necessary API functions to the game, so it can manage resource lifetimes manually.

Each resource has its own lifetime requirements:

- Some resources must be loaded when the game first starts up and must stay resident in memory for the entire duration of the game. That is, their lifetimes are effectively infinite. These are sometimes called *global resources* or *global assets*. Typical examples include the player character's mesh, materials, textures and core animations, textures and fonts used on the heads-up display, and the resources for all of the standard-issue weapons used throughout the game. Any resource that is visible or audible to the player throughout the entire game (and cannot be loaded on the fly when needed) should be treated as a global resource.
- Other resources have a lifetime that matches that of a particular game level. These resources must be in memory by the time the level is first seen by the player and can be dumped once the player has permanently left the level.
- Some resources might have a lifetime that is shorter than the duration of the level in which they are found. For example, the animations and audio clips that make up an *in-game cinematic* (a mini-movie that advances the story or provides the player with important information) might be loaded in advance of the player seeing the cinematic and then dumped once the cinematic has played.
- Some resources like background music, ambient sound effects or full-screen movies are streamed "live" as they play. The lifetime of this kind of resource is difficult to define, because each byte only persists in memory for a tiny fraction of a second, but the entire piece of music sounds like it lasts for a long period of time. Such assets are typically loaded in chunks of a size that matches the underlying hardware's requirements. For example, a music track might be read in 4 KiB chunks, because that might be the buffer size used by the low-level sound system. Only two

Event	A	B	C	D	E
Initial state	0	0	0	0	0
Level X counts incremented	1	1	1	0	0
Level X loads	(1)	(1)	(1)	0	0
Level X plays	1	1	1	0	0
Level Y counts incremented	1	2	2	1	1
Level X counts decremented	0	1	1	1	1
Level X unloads, level Y loads	(0)	1	1	(1)	(1)
Level Y plays	0	1	1	1	1

Table 7.2. Resource usage as two levels load and unload.

chunks are ever present in memory at any given moment—the chunk that is currently playing and the chunk immediately following it that is being loaded into memory.

The question of when to load a resource is usually answered quite easily, based on knowledge of when the asset is first seen by the player. However, the question of when to unload a resource and reclaim its memory is not so easily answered. The problem is that many resources are shared across multiple levels. We don't want to unload a resource when level X is done, only to immediately reload it because level Y needs the same resource.

One solution to this problem is to reference-count the resources. Whenever a new game level needs to be loaded, the list of all resources used by that level is traversed, and the reference count for each resource is incremented by one (but they are not loaded yet). Next, we traverse the resources of any unneeded levels and decrement their reference counts by one; any resource whose reference count drops to zero is unloaded. Finally, we run through the list of all resources whose reference count just went from zero to one and load those assets into memory.

For example, imagine that level X uses resources A, B and C, and that level Y uses resources B, C, D and E. (B and C are shared between both levels.) Table 7.2 shows the reference counts of these five resources as the player plays through levels X and Y. In this table, reference counts are shown in boldface type to indicate that the corresponding resource actually exists in memory, while a grey background indicates that the resource is not in memory. A reference count in parentheses indicates that the corresponding resource data is being loaded or unloaded.

7.2.2.7 Memory Management for Resources

Resource management is closely related to memory management, because we must inevitably decide *where* the resources should end up in memory once they have been loaded. The destination of every resource is not always the same. For one thing, certain types of resources must reside in video RAM (or, on the PlayStation 4, in a memory block that has been mapped for access via the high-speed “garlic” bus). Typical examples include textures, vertex buffers, index buffers and shader code. Most other resources can reside in main RAM, but different kinds of resources might need to reside within different address ranges. For example, a resource that is loaded and stays resident for the entire game (global resources) might be loaded into one region of memory, while resources that are loaded and unloaded frequently might go somewhere else.

The design of a game engine’s memory allocation subsystem is usually closely tied to that of its resource manager. Sometimes we will design the resource manager to take best advantage of the types of memory allocators we have available, or vice versa—we may design our memory allocators to suit the needs of the resource manager.

As we saw in Section 6.2.1.4, one of the primary problems facing any resource management system is the need to avoid fragmenting memory as resources are loaded and unloaded. We’ll discuss a few of the more-common solutions to this problem below.

Heap-Based Resource Allocation

One approach is to simply ignore memory fragmentation issues and use a general-purpose heap allocator to allocate your resources (like the one implemented by `malloc()` in C, or the global `new` operator in C++). This works best if your game is only intended to run on personal computers, on operating systems that support virtual memory allocation. On such a system, physical memory will become fragmented, but the operating system’s ability to map noncontiguous pages of physical RAM into a contiguous virtual memory space helps to mitigate some of the effects of fragmentation.

If your game is running on a console with limited physical RAM and only a rudimentary virtual memory manager (or none whatsoever), then fragmentation will become a problem. In this case, one alternative is to defragment your memory periodically. We saw how to do this in Section 6.2.2.2.

Stack-Based Resource Allocation

A stack allocator does not suffer from fragmentation problems, because memory is allocated contiguously and freed in an order opposite to that in which it was allocated. A stack allocator can be used to load resources if the following two conditions are met:

- The game is linear and level-centric (i.e., the player watches a loading screen, then plays a level, then watches another loading screen, then plays another level).
- Each level fits into memory in its entirety.

Presuming that these requirements are satisfied, we can use a stack allocator to load resources as follows: When the game first starts up, the global resources are allocated first. The top of the stack is then marked, so that we can free back to this position later. To load a level, we simply allocate its resources on the top of the stack. When the level is complete, we can simply set the stack top back to the marker we took earlier, thereby freeing all of the level's resources in one fell swoop without disturbing the global resources. This process can be repeated for any number of levels, without ever fragmenting memory. Figure 7.3 illustrates how this is accomplished.

A double-ended stack allocator can be used to augment this approach. Two stacks are defined within a single large memory block. One grows up from the bottom of the memory area, while the other grows down from the top. As long as the two stacks never overlap, the stacks can trade memory resources back and forth naturally—something that wouldn't be possible if each stack resided in its own fixed size block.

On *Hydro Thunder*, Midway used a double-ended stack allocator. The lower stack was used for persistent data loads, while the upper was used for temporary allocations that were freed every frame. Another way a double-ended stack allocator can be used is to ping-pong level loads. Such an approach was used at Bionic Games, Inc. for one of their projects. The basic idea is to load a compressed version of level B into the upper stack, while the currently active level A resides (in uncompressed form) in the lower stack. To switch from level A to level B, we simply free level A's resources (by clearing the lower stack) and then decompress level B from the upper stack into the lower stack. Decompression is generally much faster than loading data from disk, so this approach effectively eliminates the load time that would otherwise be experienced by the player between levels.

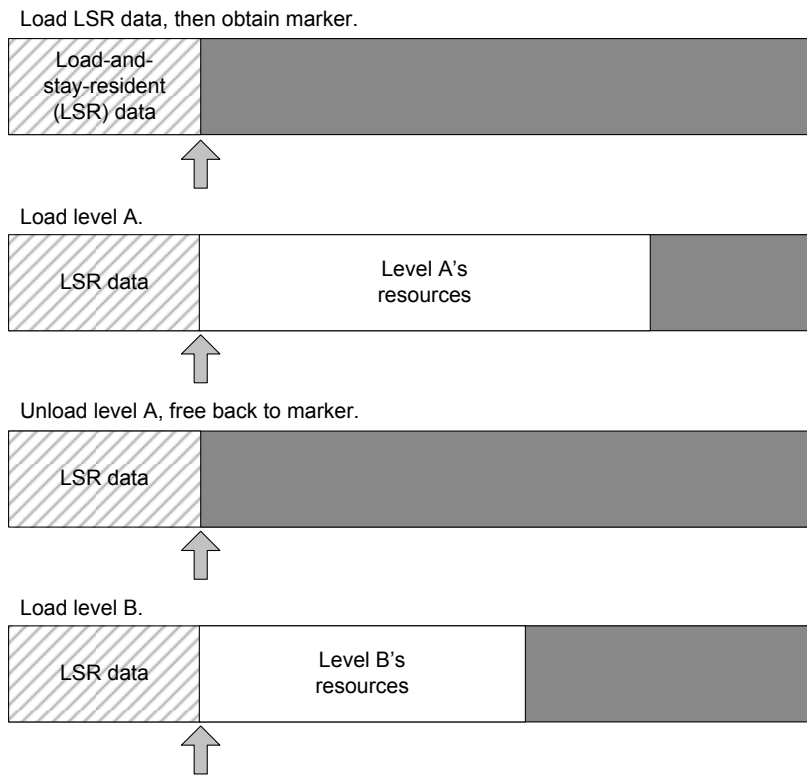


Figure 7.3. Loading resources using a stack allocator.

Pool-Based Resource Allocation

Another resource allocation technique that is common in game engines that support streaming is to load resource data in equally sized chunks. Because the chunks are all the same size, they can be allocated using a *pool allocator* (see Section 6.2.1.2). When resources are later unloaded, the chunks can be freed without causing fragmentation.

Of course, a chunk-based allocation approach requires that all resource data be laid out in a manner that permits division into equally sized chunks. We cannot simply load an arbitrary resource file in chunks, because the file might contain a contiguous data structure like an array or a very large `struct` that is larger than a single chunk. For example, if the chunks that contain an array are not arranged sequentially in RAM, the continuity of the array will be lost, and array indexing will cease to function properly. This means that all resource data must be designed with “chunkiness” in mind. Large contiguous data structures must be avoided in favor of data structures that are either small enough to fit within a single chunk or do not require contiguous RAM

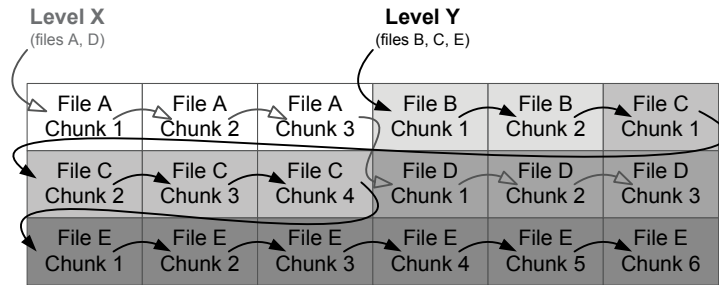


Figure 7.4. Chunky allocation of resources for levels X and Y.

to function properly (e.g., linked lists).

Each chunk in the pool is typically associated with a particular game level. (One simple way to do this is to give each level a linked list of its chunks.) This allows the engine to manage the lifetimes of each chunk appropriately, even when multiple levels with different life spans are in memory concurrently. For example, when level X is loaded, it might allocate and make use of N chunks. Later, level Y might allocate an additional M chunks. When level X is eventually unloaded, its N chunks are returned to the free pool. If level Y is still active, its M chunks need to remain in memory. By associating each chunk with a specific level, the lifetimes of the chunks can be managed easily and efficiently. This is illustrated in Figure 7.4.

One big trade-off inherent in a “chunky” resource allocation scheme is wasted space. Unless a resource file’s size is an exact multiple of the chunk size, the last chunk in a file will not be fully utilized (see Figure 7.5). Choosing a smaller chunk size can help to mitigate this problem, but the smaller the chunks, the more onerous the restrictions on the layout of the resource data. (As an extreme example, if a chunk size of one byte were selected, then no data structure could be larger than a single byte—clearly an untenable situation.) A typical chunk size is on the order of a few kibibytes. For example, at Naughty Dog, we use a chunky resource allocator as part of our resource streaming system, and our chunks are 512 KiB in size on the PS3 and 1 MiB on the PS4. You may also want to consider selecting a chunk size that is a multiple of the operating system’s I/O buffer size to maximize efficiency when loading individual chunks.

Resource Chunk Allocators

One way to limit the effects of wasted chunk memory is to set up a special memory allocator that can utilize the unused portions of chunks. As far as I’m

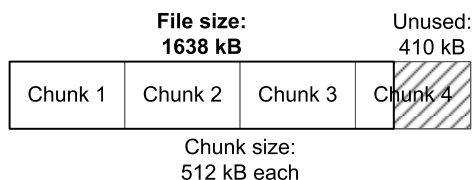


Figure 7.5. The last chunk of a resource file is often not fully utilized.

aware, there is no standardized name for this kind of allocator, but we will call it a *resource chunk allocator* for lack of a better name.

A resource chunk allocator is not particularly difficult to implement. We need only maintain a linked list of all chunks that contain unused memory, along with the locations and sizes of each free block. We can then allocate from these free blocks in any way we see fit. For example, we might manage the linked list of free blocks using a general-purpose heap allocator. Or we might map a small stack allocator onto each free block; whenever a request for memory comes in, we could then scan the free blocks for one whose stack has enough free RAM and then use that stack to satisfy the request.

Unfortunately, there's a rather grotesque-looking fly in our ointment here. If we allocate memory in the unused regions of our resource chunks, what happens when those chunks are freed? We cannot free part of a chunk—it's an all or nothing proposition. So any memory we allocate within an unused portion of a resource chunk will magically disappear when that resource is unloaded.

A simple solution to this problem is to only use our free-chunk allocator for memory requests whose lifetimes match the lifetime of the level with which a particular chunk is associated. In other words, we should only allocate memory out of level A's chunks for data that is associated exclusively with level A and only allocate from B's chunks memory that is used exclusively by level B. This requires our resource chunk allocator to manage each level's chunks separately. And it requires the users of the chunk allocator to specify which level they are allocating for, so that the correct linked list of free blocks can be used to satisfy the request.

Thankfully, most game engines need to allocate memory dynamically when loading resources, over and above the memory required for the resource files themselves. So a resource chunk allocator can be a fruitful way to reclaim chunk memory that would otherwise have been wasted.

Sectioned Resource Files

Another useful idea that is related to “chunky” resource files is the concept of *file sections*. A typical resource file might contain between one and four sections, each of which is divided into one or more chunks for the purposes of pool allocation as described above. One section might contain data that is destined for main RAM, while another section might contain video RAM data. Another section could contain temporary data that is needed during the loading process but is discarded once the resource has been completely loaded. Yet another section might contain debugging information. This debug data could be loaded when running the game in debug mode, but not loaded at all in the final production build of the game. The Granny SDK’s file system (<http://www.radgametools.com>) is an excellent example of how to implement file sectioning in a simple and flexible manner.

7.2.2.8 Composite Resources and Referential Integrity

Usually a game’s resource database consists of multiple *resource files*, each file containing one or more *data objects*. These data objects can refer to and depend upon one another in arbitrary ways. For example, a mesh data structure might contain a reference to its material, which in turn contains a list of references to textures. Usually cross-references imply dependency (i.e., if resource A refers to resource B, then both A and B must be in memory in order for the resources to be functional in the game.) In general, a game’s resource database can be represented by a *directed graph* of interdependent data objects.

Cross-references between data objects can be *internal* (a reference between two objects within a single file) or *external* (a reference to an object in a different file). This distinction is important because internal and external cross-references are often implemented differently. When visualizing a game’s resource database, we can draw dotted lines surrounding individual resource files to make the internal/external distinction clear—any edge of the graph that crosses a dotted line file boundary is an external reference, while edges that do not cross file boundaries are internal. This is illustrated in Figure 7.6.

We sometimes use the term *composite resource* to describe a self-sufficient cluster of interdependent resources. For example, a *model* is a composite resource consisting of one or more *triangle meshes*, an optional *skeleton* and an optional collection of *animations*. Each mesh is mapped with a *material*, and each material refers to one or more *textures*. To fully load a composite resource like a 3D model into memory, all of its dependent resources must be loaded as well.

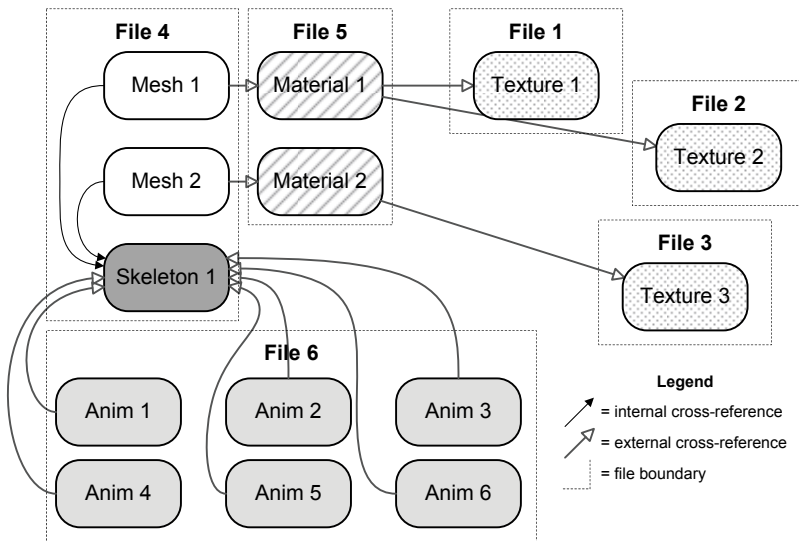


Figure 7.6. Example of a resource database dependency graph.

7.2.2.9 Handling Cross-References between Resources

One of the more-challenging aspects of implementing a resource manager is managing the cross-references between resource objects and guaranteeing that referential integrity is maintained. To understand how a resource manager accomplishes this, let's look at how cross-references are represented in memory, and how they are represented on-disk.

In C++, a cross-reference between two data objects is usually implemented via a *pointer* or a *reference*. For example, a mesh might contain the data member `Material* m_pMaterial` (a pointer) or `Material& m_material` (a reference) in order to refer to its material. However, pointers are just memory addresses—they lose their meaning when taken out of the context of the running application. In fact, memory addresses can and do change even between subsequent runs of the same application. Clearly when storing data to a disk file, we cannot use pointers to describe inter-object dependencies.

GUIDs as Cross-References

One good approach is to store each cross-reference as a string or hash code containing the unique id of the referenced object. This implies that every resource object that might be cross-referenced must have a *globally unique identifier* or *GUID*.

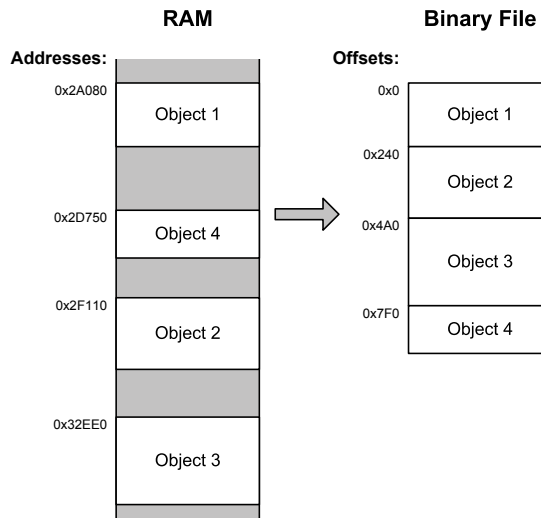


Figure 7.7. In-memory object images become contiguous when saved into a binary file.

To make this kind of cross-reference work, the runtime resource manager maintains a global resource look-up table. Whenever a resource object is loaded into memory, a pointer to that object is stored in the table with its GUID as the look-up key. After all resource objects have been loaded into memory and their entries added to the table, we can make a pass over all of the objects and convert all of their cross-references into pointers, by looking up the address of each cross-referenced object in the global resource look-up table via that object's GUID.

Pointer Fix-Up Tables

Another approach that is often used when storing data objects into a binary file is to convert the *pointers* into *file offsets*. Consider a group of C structs or C++ objects that cross-reference each other via pointers. To store this group of objects into a binary file, we need to visit each object once (and only once) in an arbitrary order and write each object's memory image into the file sequentially. This has the effect of serializing the objects into a *contiguous* image within the file, even when their memory images are not contiguous in RAM. This is shown in Figure 7.7.

Because the objects' memory images are now contiguous within the file, we can determine the *offset* of each object's image relative to the beginning of the file. During the process of writing the binary file image, we locate every

pointer within every data object, convert each pointer into an offset and store those offsets into the file in place of the pointers. We can simply overwrite the pointers with their offsets, because the offsets never require more bits to store than the original pointers. In effect, an *offset* is the binary file equivalent of a *pointer* in memory. (Do be aware of the differences between your development platform and your target platform. If you write out a memory image on a 64-bit Windows machine, its pointers will all be 64 bits wide and the resulting file won't be compatible with a 32-bit console.)

Of course, we'll need to convert the offsets back into pointers when the file is loaded into memory some time later. Such conversions are known as *pointer fix-ups*. When the file's binary image is loaded, the objects contained in the image retain their contiguous layout, so it is trivial to convert an offset into a pointer. We merely add the offset to the address of the file image as a whole. This is demonstrated by the code snippet below and illustrated in Figure 7.8.

```
U8* ConvertOffsetToPointer(U32 objectOffset,
                           U8* pAddressOfFileImage)
{
    U8* pObject = pAddressOfFileImage + objectOffset;
    return pObject;
}
```

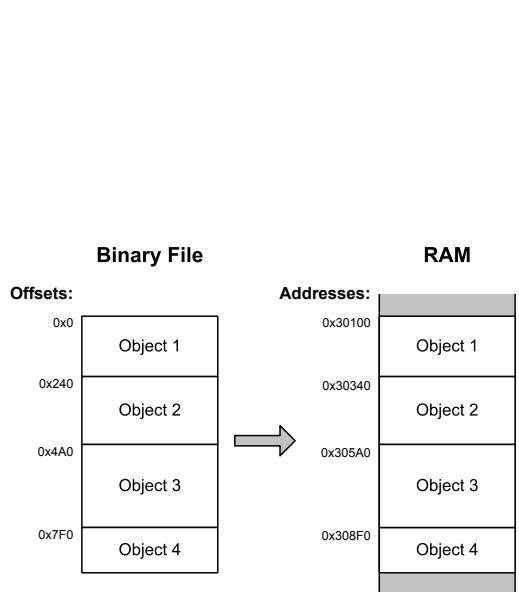


Figure 7.8. Contiguous resource file image, after it has been loaded into RAM.

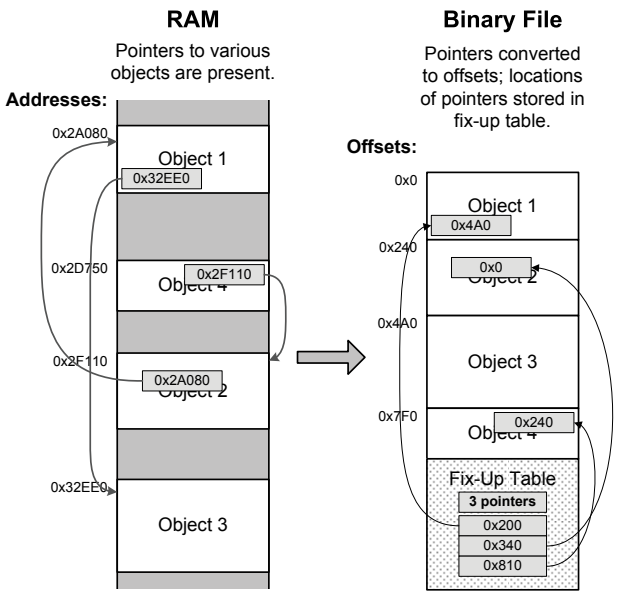


Figure 7.9. A pointer fix-up table.

The problem we encounter when trying to convert pointers into offsets, and vice versa, is how to *find* all of the pointers that require conversion. This problem is usually solved at the time the binary file is written. The code that writes out the images of the data objects has knowledge of the data types and classes being written, so it has knowledge of the locations of all the pointers within each object. The locations of the pointers are stored into a simple table known as a *pointer fix-up table*. This table is written into the binary file along with the binary images of all the objects. Later, when the file is loaded into RAM again, the table can be consulted in order to find and fix up every pointer. The table itself is just a list of offsets within the file—each offset represents a single pointer that requires fixing up. This is illustrated in Figure 7.9.

Storing C++ Objects as Binary Images: Constructors

One important step that is easy to overlook when loading C++ objects from a binary file is to ensure that the objects' constructors are called. For example, if we load a binary image containing three objects—an instance of class A, an instance of class B, and an instance of class C—then we must make sure that the correct constructor is called on each of these three objects.

There are two common solutions to this problem. First, you can simply decide not to support C++ objects in your binary files at all. In other words, restrict yourself to plain old data structures (abbreviated PODS or POD)—i.e., C structs and C++ structs and classes that contain *no virtual functions* and *trivial do-nothing constructors*. (See http://en.wikipedia.org/wiki/Plain_Old_Data_Structures for a more complete discussion of PODS.)

Second, you can save off a table containing the offsets of all non-PODS objects in your binary image along with some indication of which class each object is an instance of. Then, once the binary image has been loaded, you can iterate through this table, visit each object and call the appropriate constructor using *placement new* syntax (i.e., calling the constructor on a preallocated block of memory). For example, given the offset to an object within the binary image, we might write:

```
void* pObject = ConvertOffsetToPointer(objectOffset,
                                       pAddressOfFileImage);
::new(pObject) ClassName; // placement new syntax
```

where *ClassName* is the class of which the object is an instance.

Handling External References

The two approaches described above work very well when applied to resources in which all of the cross-references are *internal*—i.e., they only refer-

ence objects within a single resource file. In this simple case, you can load the binary image into memory and then apply the pointer fix-ups to resolve all the cross-references. But when cross-references reach out into other resource files, a slightly augmented approach is required.

To successfully represent an *external* cross-reference, we must specify not only the offset or GUID of the data object in question, but also the path to the resource file in which the referenced object resides.

The key to loading a multi-file composite resource is to load *all* of the interdependent files first. This can be done by loading one resource file and then scanning through its table of cross-references and loading any externally referenced files that have not already been loaded. As we load each data object into RAM, we can add the object's address to the master look-up table. Once all of the interdependent files have been loaded and all of the objects are present in RAM, we can make a final pass to fix up all of the pointers using the master look-up table to convert GUIDs or file offsets into real addresses.

7.2.2.10 Post-Load Initialization

Ideally, each and every resource would be completely prepared by our offline tools, so that it is ready for use the moment it has been loaded into memory. Practically speaking, this is not always possible. Many types of resources require at least some "massaging" after having been loaded in order to prepare them for use by the engine. In this book, I will use the term *post-load initialization* to refer to any processing of resource data after it has been loaded. Other engines may use different terminology. (For example, at Naughty Dog we call this *logging in* a resource.) Most resource managers also support some kind of tear-down step prior to a resource's memory being freed. (At Naughty Dog, we call this *logging out* a resource.)

Post-load initialization generally comes in one of two varieties:

- In some cases, post-load initialization is an unavoidable step. For example, on a PC, the vertices and indices that describe a 3D mesh are loaded into main RAM, but they must be transferred into video RAM before they can be rendered. This can only be accomplished at runtime, by creating a Direct X vertex buffer or index buffer, locking it, copying or reading the data into the buffer and then unlocking it.
- In other cases, the processing done during post-load initialization is avoidable (i.e., could be moved into the tools), but is done for convenience or expedience. For example, a programmer might want to add the calculation of accurate arc lengths to our engine's spline library. Rather than spend the time to modify the tools to generate the arc length data,

the programmer might simply calculate it at runtime during post-load initialization. Later, when the calculations are perfected, this code can be moved into the tools, thereby avoiding the cost of doing the calculations at runtime.

Clearly, each type of resource has its own unique requirements for post-load initialization and tear-down. So, resource managers typically permit these two steps to be configurable on a per-resource-type basis. In a non-object-oriented language like C, we can envision a look-up table that maps each type of resource to a pair of function pointers, one for post-load initialization and one for tear-down. In an object-oriented language like C++, life is even easier—we can make use of polymorphism to permit each class to handle post-load initialization and tear-down in a unique way.

In C++, post-load initialization could be implemented as a special constructor, and tear-down could be done in the class' destructor. However, there are some problems with using constructors and destructors for this purpose. For example, one typically needs to construct all loaded objects first, then apply pointer fix-ups, and finally perform post-load initialization as a separate step. As such, most developers defer post-load initialization and tear-down to plain old virtual functions. For example, we might choose to use a pair of virtual functions named something sensible like `Init()` and `Destroy()`.

Post-load initialization is closely related to a resource's memory allocation strategy, because new data is often generated by the initialization routine. In some cases, the data generated by the post-load initialization step *augments* the data loaded from the file. (For example, if we are calculating the arc lengths of the segments of a Catmull-Rom spline curve after it has been loaded, we would probably want to allocate some additional memory in which to store the results.) In other cases, the data generated during post-load initialization *replaces* the loaded data. (For example, we might allow mesh data in an older out-of-date format to be loaded and then automatically converted into the latest format for backwards compatibility reasons.) In this case, the loaded data may need to be discarded, either partially or in its entirety, after the post-load step has generated the new data.

The *Hydro Thunder* engine had a simple but powerful way of handling this. It would permit resources to be loaded in one of two ways: (a) directly into its final resting place in memory or (b) into a temporary area of memory. In the latter case, the post-load initialization routine was responsible for copying the finalized data into its ultimate destination; the temporary copy of the resource would be discarded after post-load initialization was complete. This was very useful for loading resource files that contained both relevant and irrelevant

data. The relevant data would be copied into its final destination in memory, while the irrelevant data would be discarded. For example, mesh data in an out-of-date format could be loaded into temporary memory and then converted into the latest format by the post-load initialization routine, without having to waste any memory keeping the old-format data kicking around.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

8

The Game Loop and Real-Time Simulation

Games are real-time, dynamic, interactive computer simulations. As such, *time* plays an incredibly important role in any electronic game. There are many different kinds of time to deal with in a game engine—real time, game time, the local timeline of an animation, the actual CPU cycles spent within a particular function, and the list goes on. Every engine system might define and manipulate time differently. We must have a solid understanding of all the ways time can be used in a game. In this chapter, we'll take a look at how real-time, dynamic simulation software works and explore the common ways in which time plays a role in such a simulation.

8.1 The Rendering Loop

In a graphical user interface (GUI), of the sort found on a Windows PC or a Macintosh, the majority of the screen's contents are static. Only a small part of any one window is actively changing appearance at any given moment. Because of this, graphical user interfaces have traditionally been drawn on-screen via a technique known as *rectangle invalidation*, in which only the small portions of the screen whose contents have actually changed are redrawn. Older 2D video games used similar techniques to minimize the number of pixels that needed to be drawn.